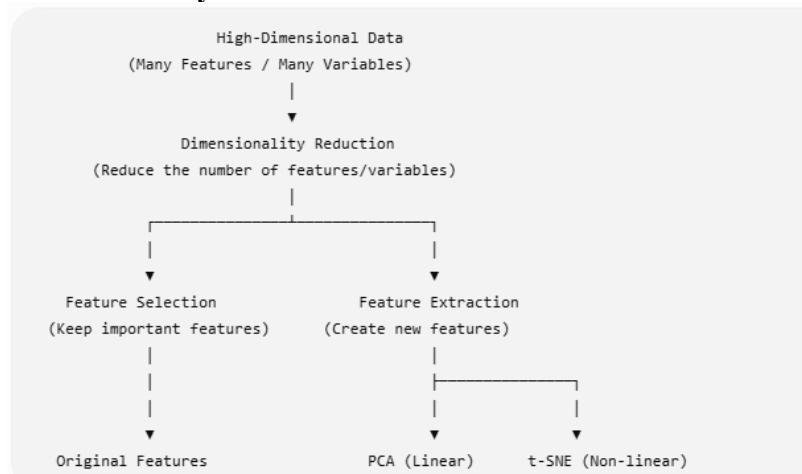


Dimensionality Reduction

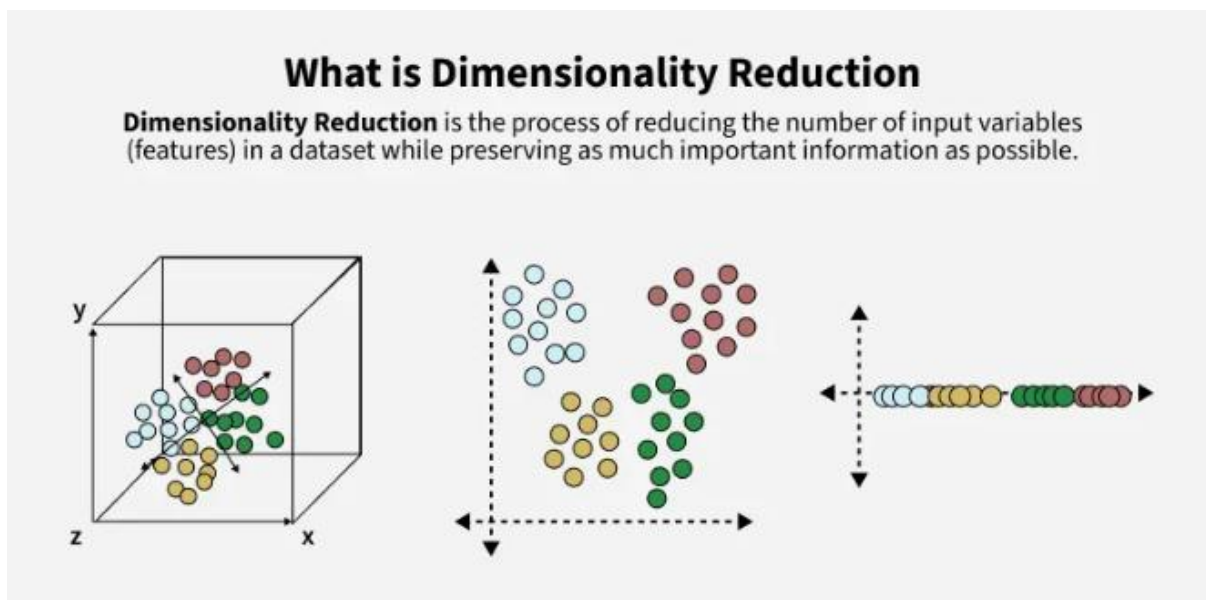


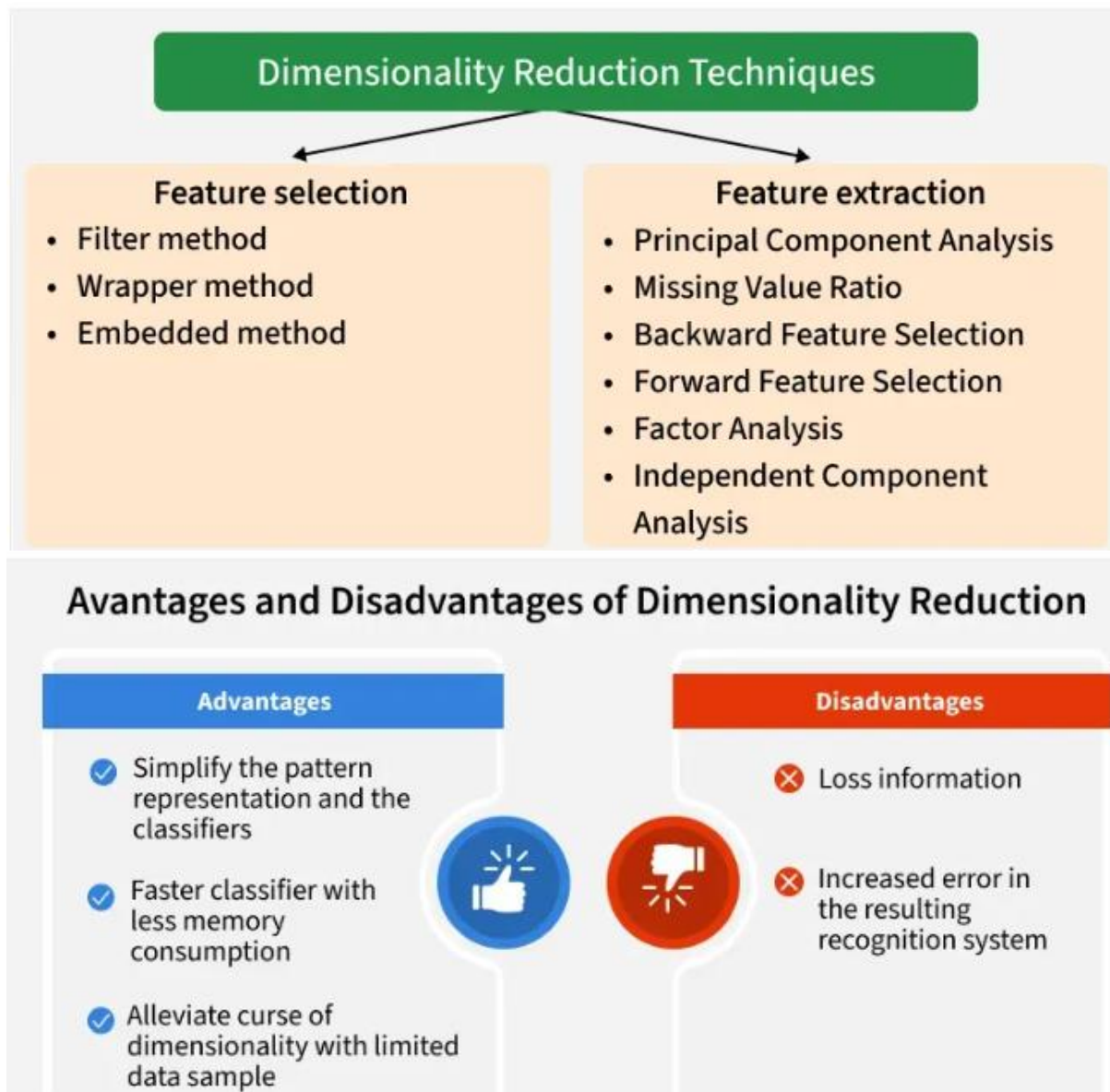
Introduction to Dimensionality Reduction

Dimensionality reduction is a technique used to reduce the number of features in a dataset while preserving important information. It transforms high-dimensional data into a lower-dimensional space for simpler representation.

- Reduces computation time by lowering the number of features
- Helps prevent overfitting by removing irrelevant data
- Improves data visualization and understanding

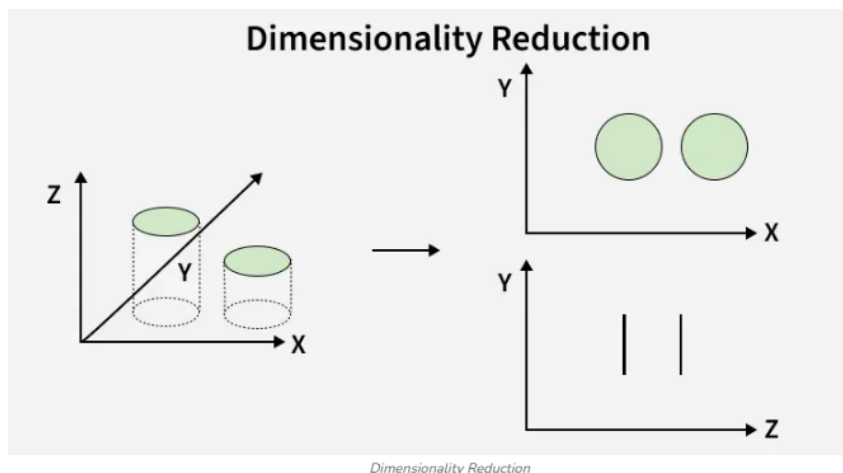
For example, when you are building a model to predict house prices with features like bedrooms, square footage and location. If you add too many features such as room condition or flooring type, the dataset becomes large and complex.





Working

Lets understand how dimensionality Reduction is used with the help of example. Imagine a dataset where each data point exists in a 3D space defined by axes X, Y and Z. If most of the data variance occurs along X and Y then the Z-dimension may contribute very little to understanding the structure of the data.



- Before Reduction we can see that data exist in 3D (X,Y,Z). It has high redundancy and Z contributes little meaningful information
- On the right after reducing the dimensionality the data is represented in lower-dimensional spaces. The top plot (X-Y) maintains the meaningful structure while the bottom plot (Z-Y) shows that the Z-dimension contributed little useful information.

This process makes data analysis more efficient hence improving computation speed and visualization while minimizing redundancy

Dimensionality Reduction Techniques

Dimensionality reduction techniques can be broadly divided into two categories:

1. Feature Selection

Feature selection chooses the most relevant features from the dataset without altering them. It helps remove redundant or irrelevant features, improving model efficiency. Some common methods are:

- **Filter methods:** Rank the features based on their relevance to the target variable.
- **Wrapper methods:** Use the model performance as the criteria for selecting features.
- **Embedded methods:** Combine feature selection with the model training process.
- **Missing Value Ratio:** Variables with missing data beyond a set threshold are removed, improving dataset reliability.
- **Backward Feature Elimination:** Starts with all features and removes the least significant ones in each iteration. The process continues until only the most impactful features remain, optimizing model performance.
- **Forward Feature Selection:** It begins with one feature, adds others incrementally and keeps those improving model performance.
- **Random Forest:** It uses decision trees to evaluate feature importance, automatically selecting the most relevant features without the need for manual coding, enhancing model accuracy.

2. Feature Extraction

Feature extraction involves creating new features by combining or transforming the original features. These new features retain most of the dataset's important information in fewer dimensions. Common feature extraction methods are:

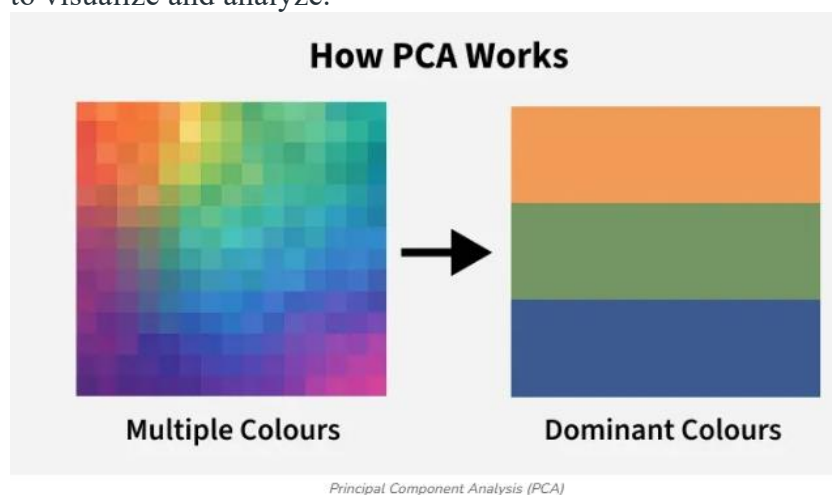
- **Principal Component Analysis (PCA):** Converts correlated variables into uncorrelated principal components hence reducing dimensionality while maintaining as much variance as possible enabling more efficient analysis.

- **Factor Analysis:** Groups variables by correlation and keeps the most relevant ones for further analysis.
- **Independent Component Analysis (ICA):** Identifies statistically independent components, ideal for applications like 'blind source separation' where traditional correlation-based methods fall short.

Principal Component Analysis (PCA)

PCA (Principal Component Analysis) is a dimensionality reduction technique and helps us to reduce the number of features in a dataset while keeping the most important information. It changes complex datasets by transforming correlated features into a smaller set of uncorrelated components.

It helps us to remove redundancy, improve computational efficiency and make data easier to visualize and analyze.



How Principal Component Analysis Works

PCA uses linear algebra to transform data into new features called principal components. It finds these by calculating eigenvectors (directions) and eigenvalues (importance) from the covariance matrix. PCA selects the top components with the highest eigenvalues and projects the data onto them to simplify the dataset.

Note: *It prioritizes the directions where the data varies the most because more variation = more useful information.*

Imagine you're looking at a messy cloud of data points like stars in the sky and want to simplify it. PCA helps you find the "most important angles" to view this cloud so you don't miss the big patterns. Here's how it works step by step:

Step 1: Standardize the Data

Different features may have different units and scales like salary vs. age. To compare them fairly PCA first standardizes the data by making each feature have:

- A mean of 0
- A standard deviation of 1

$$Z = \frac{X - \mu}{\sigma}$$

where:

- μ is the mean of independent features $\mu = \{\mu_1, \mu_2, \dots, \mu_m\}$
- σ is the standard deviation of independent features $\sigma = \{\sigma_1, \sigma_2, \dots, \sigma_m\}$

Step 2: Calculate Covariance Matrix

Next PCA calculates the **covariance matrix** to see how features relate to each other whether they increase or decrease together. The covariance between two features x_1 and x_2 is:

$$\text{cov}(x_1, x_2) = \frac{\sum_{i=1}^n (x_{1i} - \bar{x}_1)(x_{2i} - \bar{x}_2)}{n-1}$$

Where:

- $\bar{x}_1$ and $\bar{x}_2$ are the mean values of features x_1 and x_2
- n is the number of data points

The value of covariance can be positive, negative or zeros.

Step 3: Find the Principal Components

PCA identifies new axes where the data spreads out the most:

- **1st Principal Component (PC1):** The direction of maximum variance (most spread).
- **2nd Principal Component (PC2):** The next best direction, *perpendicular to PC1* and so on.

These directions come from the eigenvectors of the covariance matrix and their importance is measured by eigenvalues. For a square matrix A an eigenvector X (a non-zero vector) and its corresponding eigenvalue λ satisfy:

$$AX = \lambda X \quad AAX = \lambda AX$$

This means:

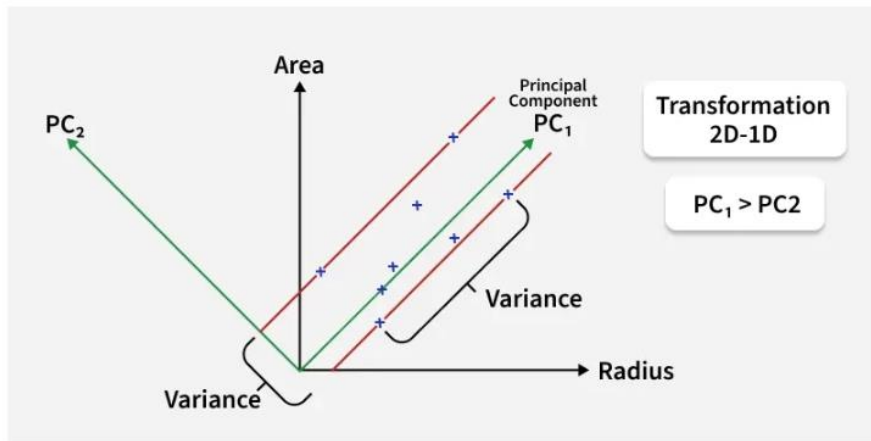
- When A acts on X it only stretches or shrinks X by the scalar λ .
- The direction of X remains unchanged hence eigenvectors define "stable directions" of A . Eigenvalues help rank these directions by importance.

Step 4: Pick the Top Directions & Transform Data

After calculating the eigenvalues and eigenvectors PCA ranks them by the amount of information they capture. We then:

1. Select the top k components that capture most of the variance like 95%.
2. Transform the original dataset by projecting it onto these top components.

This means we reduce the number of features (dimensions) while keeping the important patterns in the data.



In the above image the original dataset has two features "Radius" and "Area" represented by the black axes. PCA identifies two new directions: PC_1 and PC_2 which are the principal components.

- These new axes are rotated versions of the original ones. PC_1 captures the maximum variance in the data meaning it holds the most information while PC_2 captures the remaining variance and is perpendicular to PC_1 .
- The spread of data is much wider along PC_1 than along PC_2 . This is why PC_1 is chosen for dimensionality reduction. By projecting the data points (blue crosses) onto PC_1 we effectively transform the 2D data into 1D and retain most of the important structure and patterns.

Implementation of Principal Component Analysis in Python

Hence PCA uses a linear transformation that is based on preserving the most variance in the data using the least number of dimensions. It involves the following steps:

Step 1: Importing Required Libraries

We import the necessary library like pandas, numpy, scikit learn, seaborn and matplotlib to visualize results.

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 2: Creating Sample Dataset

We make a small dataset with three features Height, Weight, Age and Gender.

```
data = {
    'Height': [170, 165, 180, 175, 160, 172, 168, 177, 162, 158],
```

```

'Weight': [65, 59, 75, 68, 55, 70, 62, 74, 58, 54],
'Age': [30, 25, 35, 28, 22, 32, 27, 33, 24, 21],
'Gender': [1, 0, 1, 1, 0, 1, 0, 1, 0, 0] # 1 = Male, 0 = Female
}
df = pd.DataFrame(data)
print(df)

```

	Height	Weight	Age	Gender
0	170	65	30	1
1	165	59	25	0
2	180	75	35	1
3	175	68	28	1
4	160	55	22	0
5	172	70	32	1
6	168	62	27	0
7	177	74	33	1
8	162	58	24	0
9	158	54	21	0

Step 3: Standardizing the Data

Since the features have different scales Height vs Age we standardize the data. This makes all features have mean = 0 and standard deviation = 1 so that no feature dominates just because of its units.

```

X = df.drop('Gender', axis=1)
y = df['Gender']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

```

Step 4: Applying PCA algorithm

- We reduce the data from 3 features to 2 new features called principal components. These components capture most of the original information but in fewer dimensions.
- We split the data into 70% training and 30% testing sets.
- We train a logistic regression model on the reduced training data and predict gender labels on the test set.

```

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

X_train, X_test, y_train, y_test = train_test_split(X_pca, y, test_size=0.3, random_state=42)

model = LogisticRegression()

```



```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

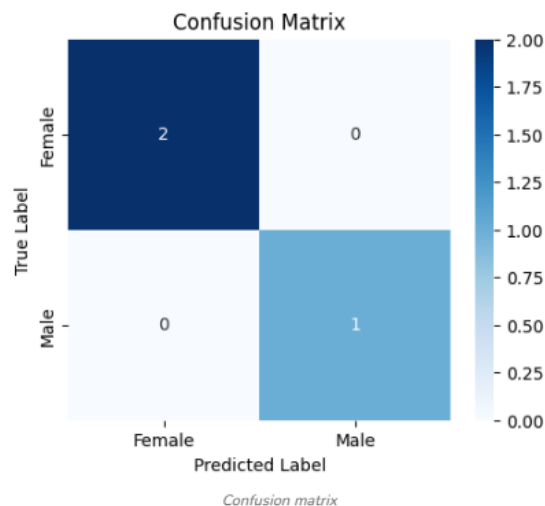
Step 5: Evaluating with Confusion Matrix

The confusion matrix compares actual vs predicted labels. This makes it easy to see where predictions were correct or wrong.

```
cm = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Female', 'Male'],
yticklabels=['Female', 'Male'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix')
plt.show()
```

Output:



Step 6: Visualizing PCA Result

```
y_numeric = pd.factorize(y)[0]
```

```
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y_numeric, cmap='coolwarm', edgecolor='k',
s=80)
plt.xlabel('Original Feature 1')
plt.ylabel('Original Feature 2')
plt.title('Before PCA: Using First 2 Standardized Features')
```

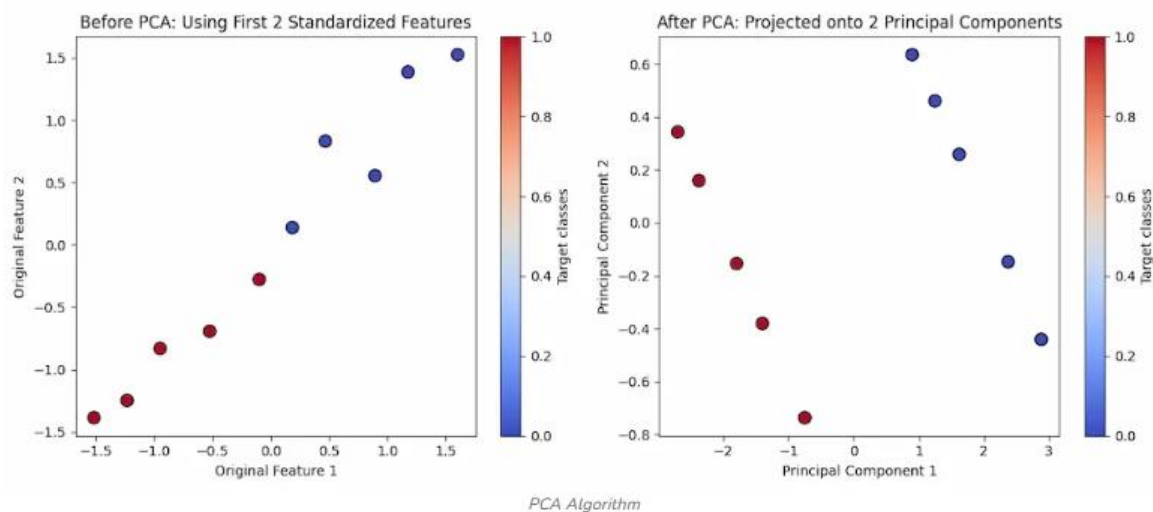


```
plt.colorbar(label='Target classes')

plt.subplot(1, 2, 2)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y_numeric, cmap='coolwarm', edgecolor='k', s=80)
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('After PCA: Projected onto 2 Principal Components')
plt.colorbar(label='Target classes')

plt.tight_layout()
plt.show()
```

Output:



- **Left Plot Before PCA:** This shows the original standardized data plotted using the first two features. There is no guarantee of clear separation between classes as these are raw input dimensions.
- **Right Plot After PCA:** This displays the transformed data using the top 2 principal components. These new components capture the maximum variance often showing better class separation and structure making it easier to analyze or model.

Feature Subset Selection Process

Feature Selection is the most critical pre-processing activity in any machine learning process. It intends to select a subset of attributes or features that makes the most meaningful contribution to a machine learning activity. In order to understand it, let us consider a small example i.e. **Predict the weight of students based on the past information about similar students**, which is captured inside a 'Student Weight' data set. The data set has 04 features like **Roll Number, Age, Height & Weight**. Roll Number has no effect on the weight of the students, so we eliminate this feature. So now the new data set will be having only 03 features. This subset of the data set is expected to give better results than the full set.

Age	Height	Weight
12	1.1	23
11	1.05	21.6
13	1.2	24.7
11	1.07	21.3
14	1.24	25.2
12	1.12	23.4

The above data set is a reduced dataset. Before proceeding further, we should look at the fact why we have reduced the dimensionality of the above dataset OR what are the issues in **High Dimensional Data**?

High Dimensional refers to the high number of variables or attributes or features present in certain data sets, more so in the domains like DNA analysis, geographic information system (GIS), etc. It may have sometimes hundreds or thousands of dimensions which is not good from the machine learning aspect because it may be a big challenge for any ML algorithm to handle that. On the other hand, a high quantity of computational and a high amount of time will be required. Also, a model built on an extremely high number of features may be very difficult to understand. **For these reasons, it is necessary to take a subset of the features instead of the full set.** So we can deduce that the objectives of feature selection are:

1. Having a faster and more cost-effective (less need for computational resources) learning model
2. Having a better understanding of the underlying model that generates the data.
3. Improving the efficacy of the learning model.

Main Factors Affecting Feature Selection

a. Feature Relevance: In the case of supervised learning, the input data set (which is the training data set), has a class label attached. A model is inducted based on the training data set — so that the inducted model can assign class labels to new, unlabeled data. Each of the predictor variables, ie expected to contribute information to decide the value of the class label. In case of a variable is not contributing any information, it is said to be irrelevant. In case the information contribution for prediction is very little, the variable is said to be weakly relevant. The remaining variables, which make a significant contribution to the prediction task are said to be strongly relevant variables.

In the case of unsupervised learning, there is no training data set or labelled data. Grouping of similar data instances are done and the similarity of data instances are evaluated based on the value of different variables. Certain variables do not contribute any useful information for

deciding the similarity of dissimilar data instances. Hence, those variable makes no significant contribution to the grouping process. These variables are marked as irrelevant variables in the context of the unsupervised machine learning task.

We can understand the concept by taking a real-world example: At the start of the article, we took a random dataset of the student. In that, Roll Number doesn't contribute any significant information in predicting what the Weight of a student would be. Similarly, if we are trying to group together students with similar academic capabilities, *Roll No* can really not contribute any information. So, in the context of grouping students with similar academic merit, the variable *Roll No* is quite irrelevant. Any feature which is irrelevant in the context of a machine learning task is a candidate for rejection when we are selecting a subset of features

b. Feature Redundancy: A feature may contribute to information that is similar to the information contributed by one or more features. For example, in the Student Data-set, both the features **Age & Height** contribute similar information. This is because, with an increase in age, weight is expected to increase. Similarly, with the increase in Height also weight is expected to increase. So, in context to that problem, Age and Height contribute similar information. In other words, irrespective of whether the feature **Height** is present or not, the learning model will give the same results. In this kind of situation where one feature is similar to another feature, **the feature is said to be potentially redundant** in the context of a machine learning problem.

All features having potential redundancy are candidates for rejection in the final feature subset. Only a few representative features out of a set of potentially redundant features are considered for being a part of the final feature subset. So in short, the main objective of feature selection is to remove all features which are irrelevant and take a representative subset of the features which are potentially redundant. This leads to a meaningful feature subset in the context of a specific learning task.

The measure of feature relevance and redundancy

a. Measures of Feature Relevance: In the case of supervised learning, **mutual information** is considered as a good measure of information contribution of a feature to decide the value of the class label. That is why it is a good indicator of the relevance of a feature with respect to the class variable. The higher the value of mutual information of a feature, the more relevant is that feature. Mutual information can be calculated as follows:

$$MI(C, f) = H(C) + H(f) - H(C, f)$$

Where, marginal entropy of the class, $H(C) = - \sum_{i=1}^k p(C_i) \log_2 p(C_i)$

Marginal entropy of the feature 'x', $H(f) = - \sum_c p(f = x) \log_2 p(f = x)$

And **K** = number of classes, **C** = class variable, **f** = feature set that take discrete values. In the case of unsupervised learning, there is no class variable. Hence, feature-to-class mutual information cannot be used to measure the information contribution of the features. In the case

of unsupervised learning, the entropy of the set of features without one feature at a time is calculated for all features. Then the features are ranked in descending order of information gain from a feature and the top ' β ' percentage (value of beta is a design parameter of the algorithm) of features are selected as relevant features. The entropy of a feature f is calculated using Shannon's formula below:

$$H(f) = - \sum_x p(f = x) \log_2 p(f = x)$$

$\sum_x$ is used only for features that take the discrete values. For continuous features, it should be replaced by discretization performed first to estimate the probabilities $p(f=x)$.

b. Measures of Feature Redundancy: There are multiple measures of similarity of information contribution, the main ones are:

- Correlation-based Measures
- Distance-based Measures
- Other coefficient-based Measure
- **1. Correlation Based Similarity Measure**
- Correlation is a measure of linear dependency between two random variables. Pearson's product correlation coefficient is one of the most popular and accepted measures correlation between two random variables. For two random feature variables F_1 and F_2 , the Pearson coefficient is defined as:

$$\sigma = \frac{cov(F_1, F_2)}{\sqrt{var(F_1) \cdot var(F_2)}}$$

$$cov(F_1, F_2) = \sum (F_{1_i} - \overline{F_1}) \cdot (F_{2_i} - \overline{F_2})$$

$$var(F_1) = \sum (F_{1_i} - \overline{F_1})^2, \quad \text{where } \overline{F_1} = \frac{1}{n} \cdot \sum F_{1_i}$$

$$var(F_2) = \sum (F_{2_i} - \overline{F_2})^2, \quad \text{where } \overline{F_2} = \frac{1}{n} \cdot \sum F_{2_i}$$

Correlation value ranges between +1 and -1. A correlation of 1 (+/-) indicates perfect correlation. In case the correlation is zero, then the features seem to have no linear relationship. Generally for all feature selection problems, a threshold value is adopted to decide whether two features have adequate similarity or not.

2. Distance-Based Similarity Measure

The most common distance measure is the **Euclidean distance**, which, between two features F_1 and F_2 are calculated as:

$$d(F_1, F_2) = \sqrt{\sum_{i=1}^n (F_{1_i} - F_{2_i})^2}$$

Where the features represent an n-dimensional dataset. Let us consider that the dataset has two features, Subjects (F_1) and marks (F_2) under consideration. The Euclidean distance between the two features will be calculated like this:

Subjects (F_1)	Marks (F_2)	$(F_1 - F_2)$	$(F_1 - F_2)^2$
2	6	-4	16
3	5.5	-2.5	6.25
6	4	2	4
7	2.5	4.5	20.25
8	3	5	25
6	5.5	0.5	0.25
6	7	-1	1
7	6	1	1
8	6	2	4
9	7	2	4

A more generalized form of the Euclidean distance is the **Minkowski Distance**, measured as $d(F_1, F_2) = \sum_{i=1}^n |F_{1i} - F_{2i}|^r$ where r is a constant.

Minkowski distance takes the form of Euclidean distance (also called **L_2 norm**) where $r = 2$. At $r=1$, it takes the form of **Manhattan** distance (also called **L_1 norm**) : $d(F_1, F_2) = \sum_{i=1}^n |F_{1i} - F_{2i}|$

3. Other Similarity Measures

Jaccard index/coefficient is used as a measure of dissimilarity between two features is complementary of Jaccard Index.

For two features having binary values, Jaccard Index is measured as: $J = \frac{n_{11}}{n_{01} + n_{10} + n_{11}}$

Where n_{11} = number of cases when both the feature have value 1,

n_{01} = number of cases where the feature 1 has value 0 and feature 2 has value 1,

n_{10} = the number of cases where feature 1 has value 1 and feature 2 has value 0.

Jaccard distance: $d_j = (1 - j)$

Let us take an example to understand it better. Consider two features, F_1 and F_2 having values (0, 1, 1, 0, 1, 0, 1, 0) and (1, 0, 0, 1, 0, 0, 0, 0).

0	1	1	0	1	0	1	0
1	1	0	0	1	0	1	0

As shown in the above picture, the cases where both the values are 0 have been left out without border- as an indication of the fact that they will be excluded in the calculation of the Jaccard coefficient.

$$\text{Jaccard coefficient of } F_1 \text{ and } F_2, J = \frac{n_{11}}{n_{01} + n_{10} + n_{11}} = \frac{2}{1+2+2} = 0.4$$

Therefore, Jaccard Distance between those two features is $d_j = (1 - 0.4) = 0.6$

Note: One more measure of similarity using similarity coefficient calculation is **Cosine Similarity**. For the sake of understanding, let us take an example of the **text classification problem**. The text needs to be first transformed into features with a word token being a feature and the number of times the word occurs in a document comes as a value in each row. There are thousands of features in such a text dataset. However, the data set is sparse in nature as only a few words do appear in a document and hence in a row of the data set. So each row has very few non-zero values. However, the non-zero values can be anything integer value as the same word may occur any number of times. Also, considering the sparsity of the dataset, the 0-0 matches need to be ignored. **Cosine similarity** which is one of the most popular measures in text classification is calculated as:

$$\cos(x, y) = \frac{x \cdot y}{\|x\| \cdot \|y\|}$$

Where, $x \cdot y$ is the vector dot product of x and $y = \sum_{i=1}^n x_i y_i$

$$\|x\| = \sqrt{\sum_{i=1}^n x_i^2} \quad \text{and} \quad \|y\| = \sqrt{\sum_{i=1}^n y_i^2}$$

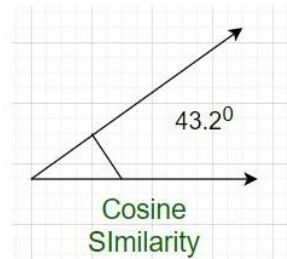
So let's calculate the cosine similarity of x and y , where $x = (2, 4, 0, 0, 2, 1, 3, 0, 0)$ and $y = (2, 1, 0, 0, 3, 2, 1, 0, 1)$. In this case, dot product of x and y will be $x \cdot y = 2*2 + 4*1 + 0*0 + 0*0 + 2*3 + 1*2 + 3*1 + 0*0 + 0*1 = 19$.

$$\|x\| = \sqrt{2^2 + 4^2 + 0^2 + 0^2 + 2^2 + 1^2 + 3^2 + 0^2 + 0^2} = \sqrt{34} = 5.83$$

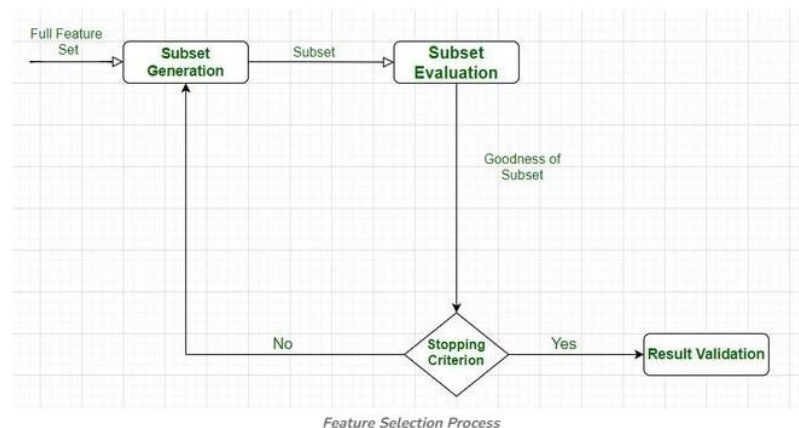
$$\|y\| = \sqrt{2^2 + 1^2 + 0^2 + 0^2 + 3^2 + 2^2 + 1^2 + 0^2 + 1^2} = \sqrt{20} = 4.47$$

$$\therefore \cos(x, y) = \frac{19}{5.83*4.47} = 0.729$$

Cosine Similarity measures the angle between x and y vectors. Hence, if cosine similarity has a value of 1, the angles between x and y is 0° which means x and y are the same except for the magnitude. If the cosine similarity is 0, the angle between x and y is 90° . Hence, they do not share any similarity. In the case of the above example, the angle comes out to be 43.2° .



Even after all these steps, there are some few more steps. You can understand it by the following flowchart:



After the successful completion of this cycle, we get the desired features, and we have finally tested them also.

The feature subset selection process involves identifying and selecting a subset of relevant features from a given dataset. It aims to improve model performance, reduce overfitting, and enhance interpretability. Here is a general outline of the feature subset selection process:

Data Preparation:

Clean the data: Handle missing values, outliers, and data inconsistencies.
Encode categorical variables: Convert categorical features into numerical representations, such as one-hot encoding or label encoding.
Normalize or standardize numerical features: Scale numerical features to a common range to avoid bias.

Feature Ranking/Scoring:

Select an appropriate feature scoring or ranking method based on the nature of the data and the problem you are addressing. Common scoring methods include correlation coefficient, information gain, chi-square test, mutual information, or statistical tests like t-test or ANOVA. Calculate the score or rank for each feature based on its relationship with the target variable.

Feature Selection Techniques:

Filter-based methods: Select features based on their individual scores or rankings. Set a threshold and select features above that threshold.

Wrapper-based methods: Use a machine learning model with different feature subsets and evaluate their performance to determine the optimal subset. Techniques like forward selection, backward elimination, or recursive feature elimination fall under this category.

Embedded methods: Incorporate feature selection as part of the model training process. Algorithms like Lasso (L1 regularization) and Ridge (L2 regularization) can automatically select relevant features during model training.

Evaluate Subset Performance:

Split the dataset into training and validation sets. Train a machine learning model using the selected subset of features. Evaluate the model's performance metrics (e.g., accuracy, precision, recall, F1-score, or area under the ROC curve) on the validation set. If the performance is not satisfactory, go back to the feature selection step and try different techniques or adjust the parameters

.

Iterative Refinement:

Iterate through steps 2-4, trying different feature scoring methods, selection techniques, thresholds, or algorithms to find the optimal feature subset. Utilize cross-validation techniques, such as k-fold cross-validation, to obtain more robust estimates of model performance and feature relevance.

Final Model Training and Testing:

Once the optimal feature subset is selected, train the final machine learning model on the entire training dataset using the selected features. Evaluate the model's performance on an independent test dataset to assess its generalization ability.

Interpretability and Validation:

Analyze the selected features and their relationship with the target variable to gain insights into the problem domain.

Validate the selected features on new, unseen data to ensure their robustness and effectiveness. Remember that the feature subset selection process is iterative and may require experimenting with different techniques, thresholds, and evaluation metrics to find the most suitable subset of features for your specific problem.

Feature Embedding in Machine Learning

. Introduction

Machine Learning algorithms work mainly with numerical data. However, real-world data such as words, sentences, images, audio, graphs, and categorical values are often complex and cannot be directly understood by many machine learning algorithms.

Feature Embedding is a technique used to represent data as a numerical vector in a continuous vector space.

The goal is to transform the original data into a representation that captures important features, relationships, similarities, and patterns.

For example:

"Cat" → [0.21, -0.45, 0.73, 0.12, ...]

"Dog" → [0.19, -0.42, 0.70, 0.15, ...]

"Car" → [-0.63, 0.82, -0.21, 0.55, ...]

The vectors representing cat and dog may be closer to each other than the vector representing car, because cat and dog have more similar meanings or characteristics.

Definition

Feature Embedding is the process of converting data or features into a numerical vector representation in a lower-dimensional or meaningful continuous vector space, where relationships between data points can be captured mathematically.

Why Do We Need Feature Embedding?

Traditional machine learning algorithms cannot directly understand complex data such as:

- Text
- Images
- Audio
- Graphs
- Categorical data

Feature embedding converts these data into numerical representations that machine learning models can process.

Example

Consider the categorical feature:

Color = {Red, Blue, Green}

A simple numerical encoding might be:

Red → 1

Blue → 2

Green → 3

But this creates an artificial ordering:

Red < Blue < Green

The model may incorrectly assume that Green is "greater than" Blue.

An embedding can instead learn a vector representation:

Red → [0.8, 0.2]

Blue → [0.1, 0.9]

Green → [0.6, 0.7]

The vector representation can capture relationships between categories.

. Basic Concepts of Feature Embedding

Vector

A vector is an ordered list of numerical values used to represent an object or data point.

Example:

Student → [0.8, 0.6, 0.9, 0.7]

Each dimension may represent some learned characteristic.

Dense Vector

A dense vector contains mostly non-zero values.

Example:

[0.25, 0.73, -0.12, 0.64, 0.91]

Dense representations are commonly used in modern machine learning and deep learning.

Embedding Space

The embedding space is the mathematical space in which data points are represented as vectors.

For example:

Dog ●

\

● Cat

● Car

Similar objects are generally positioned close together, while dissimilar objects are positioned farther apart.

The uploaded material similarly explains that embeddings represent relationships by placing similar items closer together and dissimilar items farther apart.

How Feature Embedding Works

The general process can be explained as:

Original Data

↓

Select / Learn Important Information

↓

Embedding Model

↓

Numerical Vector

↓

Embedding Space

↓

Similarity / Classification / Clustering

Step 1: Input Data

The input can be:

- Words
- Sentences
- Images
- Audio

- Graphs
- Categorical features
- Tabular data

Step 2: Embedding Model

An embedding model converts the input into a vector.

Examples:

- Word2Vec
- GloVe
- FastText
- BERT
- CNN-based models
- Autoencoders
- Node2Vec

Step 3: Vector Representation

The input is represented as a dense numerical vector.

For example:

Word: "King"

Embedding:

[0.24, -0.51, 0.72, 0.15, ...]

Step 4: Use the Embedding

The vector can then be used for:

- Similarity measurement
- Classification
- Clustering
- Recommendation
- Semantic search
- Information retrieval

Feature embedding and dimensionality reduction are related but not exactly the same.

Dimensionality Reduction

The primary goal is:

Reduce the number of dimensions while retaining important information.

Example:

100 Features



Dimensionality Reduction



10 Features

Feature Embedding

The primary goal is:

Convert data into a meaningful numerical vector representation that captures important relationships.

Example:

Word: "Computer"



Embedding



[0.25, 0.61, -0.34, 0.72, ...]

The embedding may have fewer dimensions than the original representation, but dimensionality reduction is not always its primary purpose.

Therefore:

Feature embedding can be used for dimensionality reduction, but not every embedding technique is primarily a dimensionality reduction technique.

This distinction is particularly important when teaching this topic in your Dimensionality Reduction module.

. Types of Feature Embeddings

Word Embedding

Word embedding represents individual words as numerical vectors.

Example:

King → Vector

Queen → Vector

Man → Vector

Woman → Vector

Words with related meanings tend to have similar vector representations.

Popular techniques

- Word2Vec
- GloVe
- FastText

Applications

- Sentiment analysis
- Machine translation
- Text classification
- Information retrieval

Sentence and Document Embedding

Instead of representing individual words, we can represent an entire sentence or document as a vector.

Example:

"I love machine learning"

↓

[0.23, 0.81, ...]

Applications

- Semantic search
- Document classification

- Question answering
- Text similarity

Examples include:

- BERT-based embeddings
- Sentence Transformers
- Universal Sentence Encoder

Image Embedding

Images can be converted into numerical vectors using deep learning models.

Image



CNN / Vision Model



Feature Vector

For example:

Cat Image → [0.12, 0.78, -0.23, ...]

Dog Image → [0.15, 0.74, -0.19, ...]

Similar images may have similar embeddings.

Applications

- Image classification
- Image search
- Face recognition
- Object recognition
- Image similarity

Audio Embedding

Audio signals can be transformed into vector representations.

Audio



Audio Processing Model



Audio Embedding

Applications

- Speech recognition
- Speaker identification
- Emotion detection
- Music recommendation

Graph Embedding

Graph embedding represents nodes, edges, or entire graphs as vectors.

Graph



Graph Embedding Method



Vector Representation

Examples:

- Node2Vec
- DeepWalk
- Graph Neural Networks

Applications

- Link prediction
- Node classification
- Community detection
- Recommendation systems

Categorical Feature Embedding

Categorical values can also be represented using learned vectors.

Example:

Category:

Apple

Samsung

OnePlus

Instead of assigning arbitrary numbers:

Apple → 1

Samsung → 2

OnePlus → 3

we can learn:

Apple → [0.2, 0.7, 0.1]

Samsung → [0.8, 0.3, 0.6]

OnePlus → [0.6, 0.4, 0.7]

This is widely used in:

- Recommendation systems
- Deep learning
- NLP
- Tabular machine learning

Word2Vec as an Example of Feature Embedding

For your class, I recommend using Word2Vec as the main example.

Suppose we have:

"The cat is sitting on the mat."

"The dog is sitting on the mat."

The words cat and dog occur in similar contexts.

The embedding model learns that:

Cat ↔ Dog

are semantically related.

The resulting vectors may look like:

Cat → [0.21, 0.45, 0.73, ...]

Dog → [0.23, 0.42, 0.71, ...]

Car → [-0.54, 0.81, -0.22, ...]

Therefore:

$\text{Distance}(\text{Cat}, \text{Dog}) < \text{Distance}(\text{Cat}, \text{Car})$

This demonstrates how embeddings capture relationships between data points.

Similarity in Embedding Space

Once data is represented as vectors, we can measure similarity.

One popular method is Cosine Similarity.

The basic idea is:

If two vectors point in similar directions, they are considered more similar.

Feature embeddings are used in:

1. Natural Language Processing

- Sentiment analysis
- Machine translation
- Text classification
- Semantic search

2. Recommendation Systems

- Product recommendation
- Movie recommendation
- Music recommendation

3. Computer Vision

- Image similarity
- Object recognition
- Face recognition

4. Search Engines

- Semantic search
- Similar document retrieval

5. Generative AI and RAG

Embeddings are used to represent documents and queries as vectors so that semantically relevant information can be retrieved.

Limitations

- Quality depends on the training data.
- Embeddings can inherit bias from training data.
- High-dimensional embeddings can require significant storage.
- Interpretation of individual dimensions is often difficult.
- Different embedding models may produce different representations.
- Embeddings do not always preserve every aspect of the original data.

Feature Embedding vs PCA

This comparison is essential for your module.

Feature	PCA	Feature Embedding
Main purpose	Dimensionality reduction	Meaningful vector representation
Input	Usually numerical features	Text, images, audio, categorical data, etc.
Method	Mathematical transformation	Often learned by ML/DL models
Output	Principal components	Embedding vectors
Preserves	Maximum variance	Meaning/relationships/features
Example	100 features → 10 PCs	Word → 300-dimensional vector

Feature Embedding



Convert data into numerical vectors



Represent meaningful characteristics



Similar items → Similar vectors

Dissimilar items → Different vectors



Used for

└─ Classification

└─ Clustering

└─ Similarity Search

└─ Recommendation

└─ Semantic Search

└─ Visualization

One-line definition for examination

Feature embedding is the process of transforming data into dense numerical vector representations in a continuous vector space, where important features, relationships, and similarities among data points can be captured and used by machine learning models.

Dimensionality Reduction → Introduction → Subset Selection → PCA → Feature Embedding → Factor Analysis → SVD → MDS

For Feature Embedding, teach Word2Vec as the main example, briefly introduce text, image, audio, graph, and categorical embeddings, and then explain t-SNE only as a method for visualizing high-dimensional embeddings. This will connect the topic very nicely to the rest of your Dimensionality Reduction module.

Visualization using t-SNE

t-SNE is used to visualize high dimensional word embeddings by reducing them to 2D space, helping us understand how similar words are positioned relative to each other.

Step 1: Import Libraries

- **NumPy:** Handles numerical data and array manipulation.
- **Matplotlib:** Creates plots and visualizations.
- **scikit-learn:** Reduces high-dimensional vectors to two dimensions for easy visualization.
- **Gensim:** Downloads text datasets and trains word embedding models.

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.manifold import TSNE
```

```
import gensim.downloader as api
```

```
from gensim.models import Word2Vec
```

Step 2: Load Data and Train Word2Vec Model

Loads a sample text dataset and uses it to train a Word2Vec model which creates word vectors.

```
corpus = api.load('text8')
```

```
model = Word2Vec(corpus)
```

Step 3: Select Words and Get Their Embeddings

- Chooses a list of sample words.
- Extracts their vector representations from the model as NumPy arrays.

```
words = ['cat', 'dog', 'elephant', 'lion', 'bird', 'rat', 'wolf', 'cow',  
         'goat', 'snake', 'rabbit', 'human', 'parrot', 'fox', 'peacock',  
         'lotus', 'roses', 'marigold', 'jasmine', 'computer', 'robot',  
         'software', 'vocabulary', 'machine', 'eye', 'vision',  
         'grammar', 'words', 'sentences', 'language', 'verbs', 'noun',  
         'transformer', 'embedding', 'neural', 'network', 'optimization']  
words = [word for word in words if word in model.wv.key_to_index]  
word_embeddings = [model.wv[word] for word in words]  
embeddings = np.array(word_embeddings)
```

Step 4: Reduce Dimensionality with t-SNE

Uses t-SNE from scikit learn to shrink high dimensional word vectors into two dimensions for visualization.

```
tsne = TSNE(n_components=2, perplexity=2)  
embeddings_2d = tsne.fit_transform(embeddings)
```

Step 5: Plot Embedding

Displays a scatter plot of the words in 2D space, labels each point with its word and displays the plot.

```
plt.figure(figsize=(10, 7), dpi=1000)  
plt.scatter(embeddings_2d[:, 0], embeddings_2d[:, 1], marker='o')  
for i, word in enumerate(words):  
    plt.text(embeddings_2d[i, 0], embeddings_2d[i, 1],
```

```

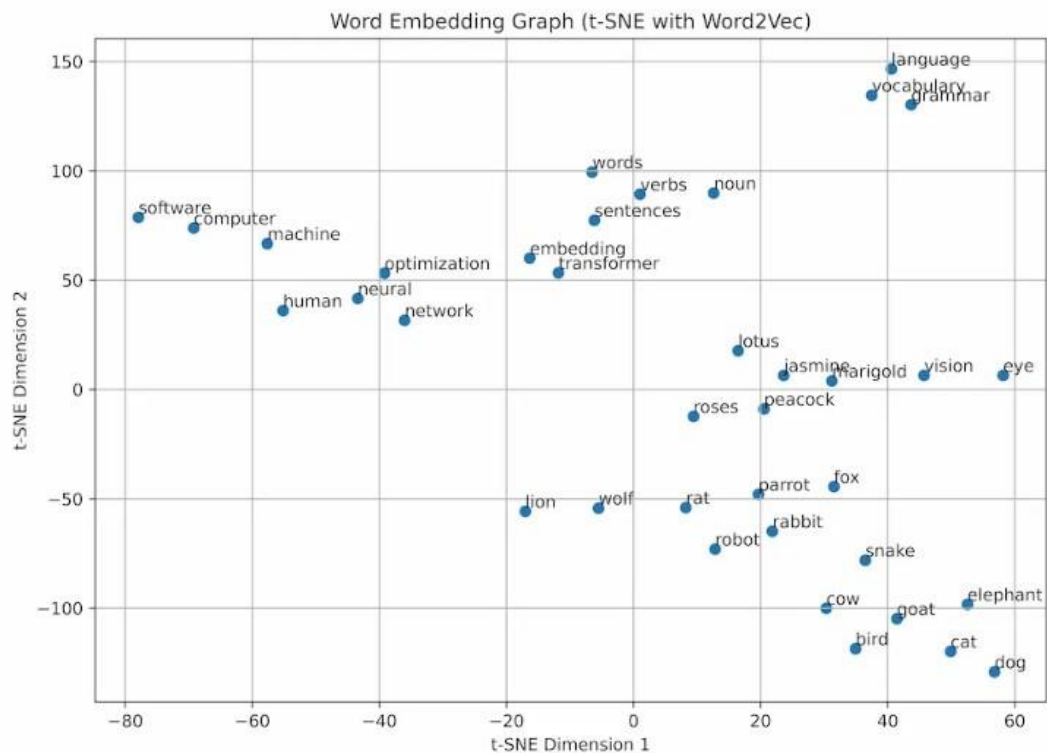
word, fontsize=10, ha='left', va='bottom')

plt.xlabel('t-SNE Dimension 1')
plt.ylabel('t-SNE Dimension 2')
plt.title('Word Embedding Graph (t-SNE with Word2Vec)')
plt.grid(True)
plt.savefig('embedding.png')
plt.show()

```

Output:

Original embedding vector shape (37, 100)
After applying t-SNE embedding vector shape (37, 2)



Output:

Factor Analysis

Factor Analysis is a statistical technique used in data analysis to identify hidden patterns or underlying relationships among a large set of variables. It helps reduce data complexity by grouping correlated variables into smaller sets called factors which represent shared characteristics or dimensions within the data.

actor analysis serves several purposes and objectives in statistical analysis:

1. **Dimensionality Reduction:** Simplifies datasets by grouping correlated variables into fewer factors making data easier to interpret.
2. **Identifying Latent Constructs:** Reveals hidden variables like traits, attitudes that explain observed patterns.
3. **Data Summarization:** Summarizes many variables into a concise set of factors while retaining key information.
4. **Hypothesis Testing:** Evaluates whether the data support expected relationships among variables.
5. **Variable Selection:** Highlights the most relevant variables for analysis or modelling.
6. **Improving Models:** Reduces multicollinearity and enhances predictive model performance.

Commonly Used Terms in Factor Analysis

Most commonly used terms in factor analysis are:

1. **Factors:** Hidden variable that explains patterns among observed variables.
2. **Factor Loading:** Strength of the relationship between a variable and a factor.
3. **Eigenvalue:** Amount of variance explained by each factor.
4. **Communalities:** Explains how much of a variable's variance is explained by the extracted factors.
5. **Rotation:** Technique to make factors more interpretable like Varimax.
6. **Scree Plot:** A plot used to determine the number of factors to retain based on the magnitude of eigenvalues.
7. **Kaiser-Meyer-Olkin (KMO) Measure:** It checks if your data is suitable for factor analysis, values closer to 1 mean the data is a good fit.

Singular Value Decomposition (SVD)

Singular Value Decomposition (SVD) is a factorization method in linear algebra that decomposes a matrix into three other matrices, providing a way to represent data in terms of its singular values.

SVD helps you split that table into three parts:

- **U:** This part tells you about the people (like their general preferences).
- **Σ :** This part shows how important each factor is (how much each rating matters).

- V^T : This part tells you about the products (how similar they are to each other)

Lets understand this with help of an example: Suppose you have a small table of people's ratings for two movies,

Name	Movie 1 Rating	Movie 2 Rating
Amit	5	3
Sanket	4	2
Harsh	2	5

- SVD breaks this table into three smaller parts: one that shows people's preferences, one that shows the importance of each movie, and one that shows how similar the movies are to each other
- Mathematically, the SVD of a matrix A (of size $m \times n$) is represented as: $A = U \Sigma V^T$

Here:

- U : An $m \times m$ orthogonal matrix whose columns are the left singular vectors of A .
- Σ : A diagonal $m \times n$ matrix containing the singular values of A in descending order.
- V^T : The transpose of an $n \times n$ orthogonal matrix, where the columns are the right singular vectors of A .

How to perform Singular Value Decomposition

To perform Singular Value Decomposition (SVD) for the matrix

$$A = \begin{bmatrix} 3 & 2 & 2 \\ 2 & 3 & -2 \end{bmatrix},$$

Step 1: Compute AA^T

First, we need to calculate the matrix AA^T (where A^T is the transpose of matrix A):

$$A = \begin{bmatrix} 3 & 2 & 2 \\ 2 & 3 & -2 \end{bmatrix}$$

$$A^T = \begin{bmatrix} 3 & 2 \\ 2 & 3 \\ 2 & -2 \end{bmatrix}$$

Now, compute AA^T :

$$AA^T = \begin{bmatrix} 3 & 2 & 2 \\ 2 & 3 & -2 \end{bmatrix} \cdot \begin{bmatrix} 3 & 2 \\ 2 & 3 \\ 2 & -2 \end{bmatrix} = \begin{bmatrix} 17 & 8 \\ 8 & 17 \end{bmatrix}$$

Step 2: Find the Eigenvalues of AA^T

To find the eigenvalues of AA^T , we solve the characteristic equation:

$$\det(AA^T - \lambda I) = 0$$

$$\det \begin{bmatrix} 17 - \lambda & 8 \\ 8 & 17 - \lambda \end{bmatrix} = 0$$

$$(\lambda - 25)(\lambda - 9) = 0$$

Thus, the eigenvalues are $\lambda_1=25$ and $\lambda_2=9$. These eigenvalues correspond to the singular values $\sigma_1=5$ and $\sigma_2=3$, since the singular values are the square roots of the eigenvalues.

Step 3: Find the Right Singular Vectors (Eigenvectors of $A^T A$)

Next, we find the eigenvectors of $A^T A$ for $\lambda = 25$ and $\lambda = 9$.

For $\lambda = 25$:

Solve $(A^T A - 25I)v = 0$:

$$A^T A - 25I = \begin{bmatrix} -12 & 12 & 2 \\ 12 & -12 & -2 \\ 2 & -2 & -17 \end{bmatrix}$$

Row-reduce this matrix to:

$$\begin{bmatrix} 1 & -1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

The eigenvector corresponding to $\lambda = 25$ is:

$$v_1 = \begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \end{bmatrix}$$

For $\lambda = 9$:

Solve $(A^T A - 9I)v = 0$:

The eigenvector corresponding to $\lambda = 9$ is:

$$v_2 = \begin{bmatrix} \frac{1}{\sqrt{18}} \\ \frac{\sqrt{18}}{-1} \\ \frac{\sqrt{18}}{4} \\ \frac{\sqrt{18}}{\sqrt{18}} \end{bmatrix}$$

For the third eigenvector v_3 :

Since v_3 must be perpendicular to v_1 and v_2 , we solve the system $v_1^T v_3 = 0$ and $v_2^T v_3 = 0$, leading to:

$$v_3 = \begin{bmatrix} \frac{2}{3} \\ \frac{3}{2} \\ \frac{-1}{3} \\ 3 \end{bmatrix}$$

Step 4: Compute the Left Singular Vectors (Matrix U)

To compute the left singular vectors U , we use the formula $u_i = \frac{1}{\sigma_i} A v_i$. This results in:

$$U = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} \end{bmatrix}$$

Step 5: Final SVD Equation

Finally, the Singular Value Decomposition of matrix A is:

$$A = U \Sigma V^T$$

Where:

$$U = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} \end{bmatrix}$$

$$\Sigma = \begin{bmatrix} 5 & 0 & 0 \\ 0 & 3 & 0 \end{bmatrix}$$

$$V = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{18}} & \frac{-1}{\sqrt{18}} & \frac{4}{\sqrt{18}} \\ \frac{2}{3} & \frac{-2}{3} & \frac{1}{3} \end{bmatrix}$$

Thus, the SVD of matrix A is:

$$A = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} \end{bmatrix} \begin{bmatrix} 5 & 0 & 0 \\ 0 & 3 & 0 \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{18}} & \frac{-1}{\sqrt{18}} & \frac{4}{\sqrt{18}} \\ \frac{2}{3} & \frac{-2}{3} & \frac{1}{3} \end{bmatrix}$$

This is the Result SVD matrix of matrix A.

Implementation of Singular Value Decomposition (SVD)

In this code, we will try to calculate the Singular value decomposition using Numpy and Scipy. We will be calculating SVD, and also performing pseudo-inverse. In the end, we can apply SVD for compressing the image

```
from skimage.color import rgb2gray
```

```
from skimage import data
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
from scipy.linalg import svd
```

```
X = np.array([[3, 3, 2], [2, 3, -2]])
```

```
print(X)
```

```
U, singular, V_transpose = svd(X)
```

```

print("U: ", U)
print("Singular array", singular)
print("V^{T}", V_transpose)
singular_inv = 1.0 / singular
s_inv = np.zeros(X.shape)
s_inv[0][0] = singular_inv[0]
s_inv[1][1] = singular_inv[1]
M = np.dot(np.dot(V_transpose.T, s_inv.T), U.T)
print(M)
cat = data.chelsea()
plt.imshow(cat)
gray_cat = rgb2gray(cat)
U, S, V_T = svd(gray_cat, full_matrices=False)
S = np.diag(S)
fig, ax = plt.subplots(5, 2, figsize=(8, 20))
curr_fig = 0
for r in [5, 10, 70, 100, 200]:
    cat_approx = U[:, :r] @ S[0:r, :r] @ V_T[:, :r]
    ax[curr_fig][0].imshow(cat_approx, cmap='gray')
    ax[curr_fig][0].set_title("k = " + str(r))
    ax[curr_fig, 0].axis('off')
    ax[curr_fig][1].set_title("Original Image")
    ax[curr_fig][1].imshow(gray_cat, cmap='gray')
    ax[curr_fig, 1].axis('off')
    curr_fig += 1
plt.show()

```

Output:

```
[[ 3  3  2]
 [ 2  3 -2]]
-----
U: [[-0.7815437 -0.6238505]
     [-0.6238505  0.7815437]]
-----
Singular array [5.54801894 2.86696457]
-----
V^{T} [[-0.64749817 -0.7599438 -0.05684667]
        [-0.10759258  0.16501062 -0.9804057 ]
        [-0.75443354  0.62869461  0.18860838]]
-----
# Inverse
array([[ 0.11462451,  0.04347826],
       [ 0.07114625,  0.13043478],
       [ 0.22134387, -0.26086957]])
-----
```

Multidimensional Scaling

Multidimensional Scaling (MDS) is a statistical tool that helps discover the connections among objects in lower dimensional space using the canonical similarity or dissimilarity data analysis technique. The article aims to delve into the fundamentals of multidimensional scaling.

Understanding Multidimensional Scaling (MDS)

Multidimensional Scaling (MDS) is a statistical technique that visualizes the similarity or dissimilarity among a set of objects or entities by translating high-dimensional data into a more comprehensible two- or three-dimensional space. This reduction aims to maintain the inherent relationships within the data, facilitating easier analysis and interpretation. MDS is particularly useful in fields such as psychology, sociology, marketing, geography, and biology, where understanding complex structures is crucial for decision-making and strategic planning.

Multidimensional Scaling (MDS) in Python

Import the NumPy and sklearn.manifold.MDS modules and then generate some random data with 10 dimensions and 100 samples.

```
import numpy as np
from sklearn.manifold import MDS
# Generate some random data with
# 10 dimensions and 100 samples
X = np.random.randn(100, 10)
```

Create an MDS object with 2 dimensions and a random start. Fit the data to the MDS object and transform the data.

```
# Create an MDS object with  
# 2 dimensions and random start  
mds = MDS(n_components=2, random_state=0)  
  
# Fit the data to the MDS  
# object and transform the data  
X_transformed = mds.fit_transform(X)
```

Print the shape of the transformed data.

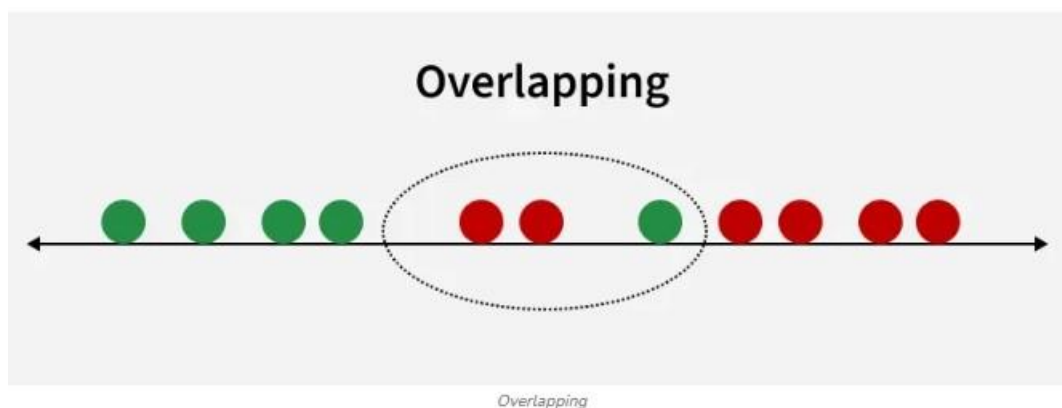
```
# Print the shape of the transformed data  
print(X_transformed.shape)
```

Output:

```
(100, 2)
```

Linear Discriminant Analysis in Machine Learning

Linear Discriminant Analysis (LDA) also known as Normal Discriminant Analysis is supervised classification problem that helps separate two or more classes by converting higher-dimensional data space into a lower-dimensional space. It is used to identify a linear combination of features that best separates classes within a dataset.



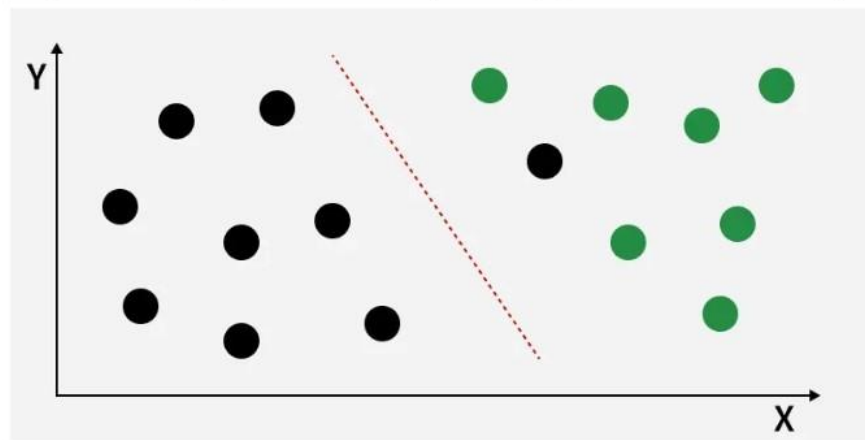
For example we have two classes that need to be separated efficiently. Each class may have multiple features and using a single feature to classify them may result in overlapping. To solve this LDA is used as it uses multiple features to improve classification accuracy. LDA works by some assumptions and we are required to understand them so that we have a better understanding of its working.

Key Assumptions of LDA

For LDA to perform effectively, certain assumptions are made:

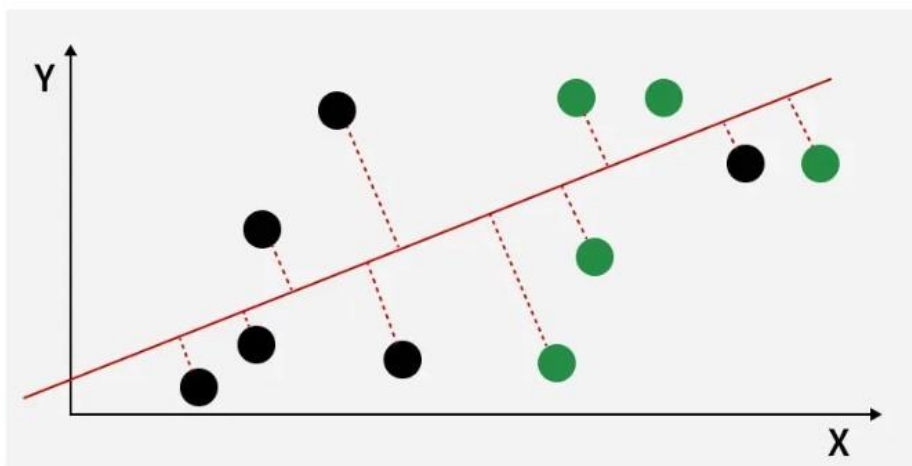
- **Gaussian Distribution:** The data in each class should follow a normal bell-shaped distribution.
- **Equal Covariance Matrices:** All classes should have the same covariance structure.
- **Linear Separability:** The data should be separable using a straight line or plane.

LDA can produce very good results if it meets these assumptions. For example when data points belonging to two classes are plotted, if they are not linearly separable LDA will attempt to find a projection that maximizes class separability.



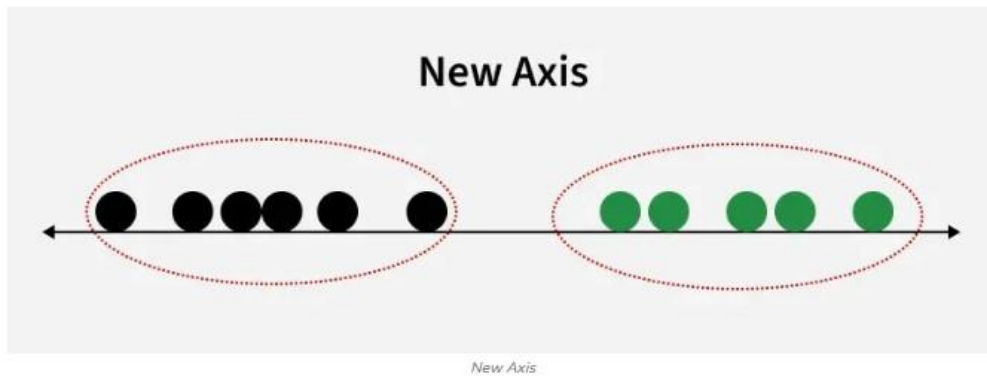
Linearly Separable Dataset

The image shows classes (black and green) that are not linearly separable. LDA finds a new axis (red dashed line) that maximizes the distance between class means while minimizing within-class variance, improving class separation for better classification.



The perpendicular distance between the line and points

The perpendicular distance from the decision boundary to the data points shows how LDA reduces within-class variation and increases class separability. The data points are then projected onto the new axis, as shown in the figure below.



This shows how LDA creates a new axis to project the data and separate two classes along a linear path. However, when class distributions share the same mean, LDA cannot find a separating axis and non-linear discriminant analysis is needed.

Separating Hyperplanes in SVM

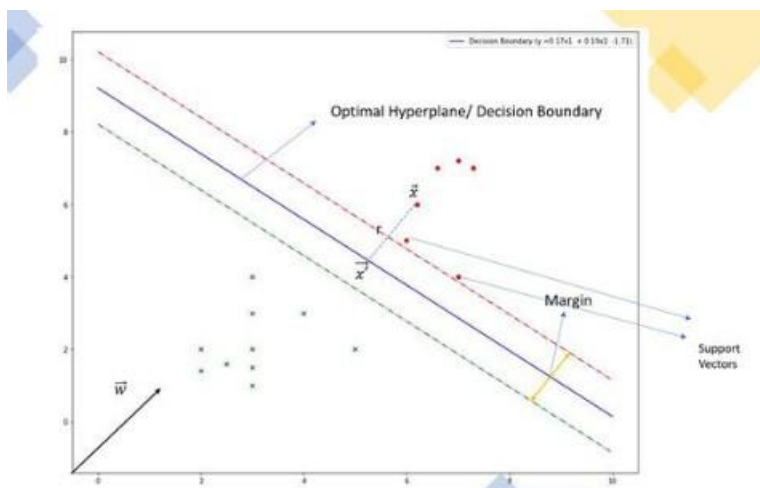
Support Vector Machine is the supervised machine learning algorithm, that is used in both classification and regression of models. The idea behind it is simple to just find a plane or a boundary that separates the data between two classes.

Support Vectors:

Support vectors are the data points that are close to the decision boundary, they are the data points most difficult to classify, they hold the key for SVM to be optimal decision surface. The optimal hyperplane comes from the function class with the lowest capacity i.e minimum number of independent features/parameters.

Separating Hyperplanes:

Below is an example of a scatter plot:



In the above scatter, Can we find a line that can separate two categories. Such a line is called separating hyperplane. So, why it is called a hyperplane, because in 2-dimension, it's a line but for 1-dimension it can be a point, for 3-dimension it is a plane, and for 3 or more dimensions it is a hyperplane

Now, we understand the hyperplane, we also need to find the most optimized hyperplane. The idea behind that this hyperplane should be farthest from the support vectors. This distance b/w separating hyperplanes and support vector known as margin. Thus, the best hyperplane will be whose margin is the maximum.

Generally, the margin can be taken as $2 \cdot p$, where p is the distance b/w separating hyperplane and nearest support vector. Below is the method to calculate linearly separable hyperplane.

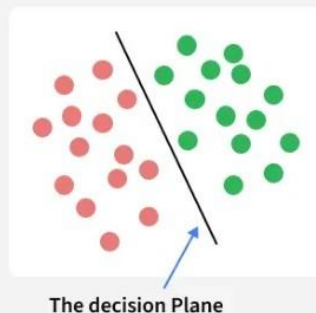
A separating hyperplane can be defined by two terms: an intercept term called b and a decision hyperplane normal vector called w . These are commonly referred to as the weight vector in machine learning. Here b is used to select the hyperplane i.e perpendicular to the normal vector. Now since all the plane x in the hyperplane should satisfy the following equation:

$$w^T \cdot x = -b$$

Support Vector Machine (SVM) Algorithm

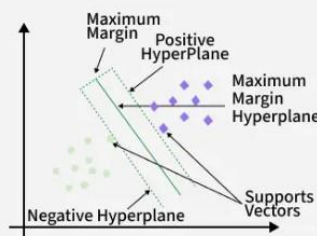
Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression tasks. It tries to find the best boundary known as hyperplane that separates different classes in the data. It is useful when you want to do binary classification like spam vs. not spam or cat vs. dog.

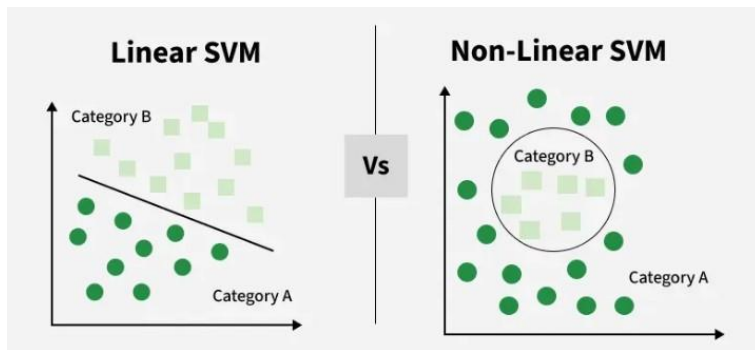
- Support Vector Machine is used for classification and regression.
- It works by finding decision planes that separate data into different classes.



Support Vectors & Hyperplane

- Support Vectors are the closest data points to the hyperplane that define the class boundary.
- A hyperplane is a plane that separates different classes.





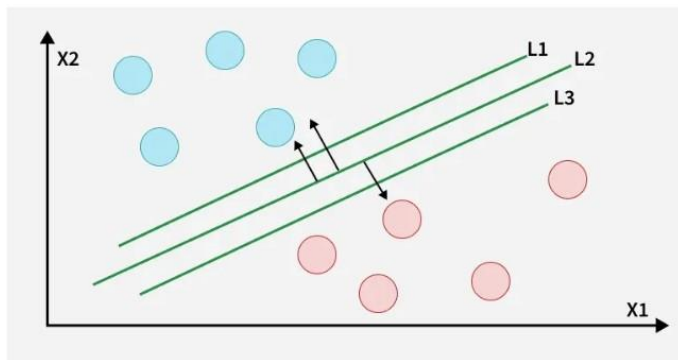
The main goal of SVM is to maximize the margin between the two classes. The larger the margin the better the model performs on new and unseen data.

Key Concepts

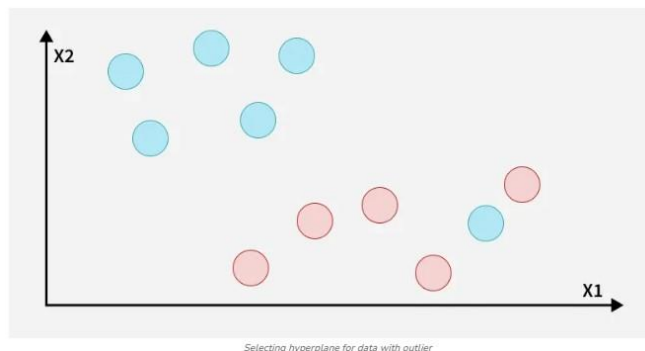
- **Hyperplane:** A decision boundary separating different classes in feature space and is represented by the equation $wx + b = 0$ in linear classification.
- **Support Vectors:** The closest data points to the hyperplane, crucial for determining the hyperplane and margin in SVM.
- **Margin:** The distance between the hyperplane and the support vectors. SVM aims to maximize this margin for better classification performance.
- **Kernel:** A function that maps data to a higher-dimensional space enabling SVM to handle non-linearly separable data.
- **Hard Margin:** A maximum-margin hyperplane that perfectly separates the data without misclassifications.
- **Soft Margin:** Allows some misclassifications by introducing slack variables, balancing margin maximization and misclassification penalties when data is not perfectly separable.
- **C:** A regularization term balancing margin maximization and misclassification penalties. A higher C value forces stricter penalty for misclassifications.
- **Hinge Loss:** A loss function penalizing misclassified points or margin violations and is combined with regularization in SVM.
- **Dual Problem:** Involves solving for Lagrange multipliers associated with support vectors, facilitating the kernel trick and efficient computation.

Working of Support Vector Machine Algorithm

The key idea behind the SVM algorithm is to find the hyperplane that best separates two classes by maximizing the margin between them. This margin is the distance from the hyperplane to the nearest data points (support vectors) on each side.



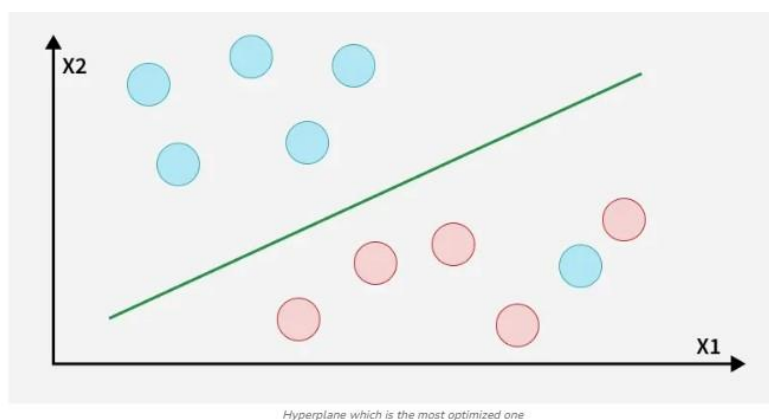
The best hyperplane also known as the hard margin is the one that maximizes the distance between the hyperplane and the nearest data points from both classes. This ensures a clear separation between the classes. So from the above figure, we choose L2 as hard margin. Let's consider a scenario like shown below:



Here, we have one blue ball in the boundary of the red ball.

How does SVM classify the data?

The blue ball in the boundary of red ones is an outlier of blue balls. The SVM algorithm has the characteristics to ignore the outlier and finds the best hyperplane that maximizes the margin. SVM can be sensitive to outliers, especially in the case of a hard margin, while soft margin SVM helps reduce their impact by allowing some misclassifications.



A soft margin allows for some misclassifications or violations of the margin to improve generalization. The SVM optimizes the following equation to balance margin maximization and penalty minimization:

$$\text{Objective Function} = \left(\frac{1}{\text{margin}}\right) + \lambda \sum \text{penalty}$$

The penalty used for violations is often hinge loss which has the following behavior:

- If a data point is correctly classified and lies outside the margin, there is no penalty (loss = 0). If it is correctly classified but lies within the margin or is misclassified, the hinge loss is greater than 0.
- If a point is incorrectly classified or violates the margin the hinge loss increases proportionally to the distance of the violation.

Kernel Trick in Support Vector Classification

Support Vector Machines (SVMs) have proven to be a powerful and versatile tool for classification tasks. A key component that significantly enhances the capabilities of SVMs, particularly in dealing with non-linear data, is the Kernel Trick. This article delves into the intricacies of the Kernel Trick, its motivation, implementation, and practical applications.

Linear vs Non-Linear Problems

For linearly separable data, finding this hyperplane is straightforward. However, many real-world problems are non-linear, meaning that no linear separation can perfectly divide the classes. This is where the kernel trick comes into play.

Concept of Feature Mapping

To deal with non-linear data, one approach could be to map the input data into a higher-dimensional space where it is linearly separable. This mapping involves transforming the data into a new space (feature space) where the separation between the data points is clearer.

For example, consider a set of data points that are not linearly separable in two dimensions. By mapping these points into a three-dimensional space, we might find that they can be separated by a plane in this higher-dimensional space.

What is the Kernel Trick?

The kernel trick is a method used in SVMs to enable them to classify non-linear data using a linear classifier. By applying a kernel function, SVMs can implicitly map input data into a higher-dimensional space where a linear separator (hyperplane) can be used to divide the classes. This mapping is computationally efficient because it avoids the direct calculation of the coordinates in this higher space.

Types of Kernel Functions

Several kernel functions can be used, each suited to different types of data distributions:

- **Linear Kernel:** No mapping is needed as the data is already assumed to be linearly separable.
- **Polynomial Kernel:** Maps inputs into a polynomial feature space, enhancing the classifier's ability to capture interactions between features.
- **Radial Basis Function (RBF) Kernel:** Also known as the Gaussian kernel, it is useful for capturing complex regions by considering the distance between points in the input space.
- **Sigmoid Kernel:** Mimics the behavior of neural networks by using a sigmoid function as the kernel.

Multi-class classification using Support Vector Machines (SVM)

Support Vector Machines (SVM) are widely recognized for their effectiveness in binary classification tasks. However, real-world problems often require distinguishing between more than two classes. This is where multi-class classification comes into play. While SVMs are inherently binary classifiers, they can be extended to handle multi-class classification problems. This article explores the techniques used to adapt SVMs for multi-class tasks, the challenges involved, and how to implement multi-class SVMs using scikit-learn.

Understanding Multi-Class Classification

In multi-class classification, the goal is to categorize an instance into one of three or more classes. For example, consider a problem where you need to classify an image as either a cat, dog, or bird. Unlike binary classification, which deals with two classes, multi-class classification must handle multiple possible outcomes.

Multi-Class Classification With SVMs

SVMs are designed to separate data points into two distinct classes by finding the optimal hyperplane that maximizes the margin between the classes. This binary nature makes SVMs particularly well-suited for two-class problems. However, when there are more than two classes, SVMs need to be adapted to handle the additional complexity. **Multi-class Classification Challenges:**

- In binary classification, SVMs are straightforward as they separate data into two classes.
- However, multi-class classification involves more complexity since there are more than two classes to consider. The challenge is to extend the binary classification capability of SVMs to handle multiple classes effectively.

Strategies for Multi-class Classification with SVM

When dealing with multi-class classification using Support Vector Machines (SVM), two primary strategies are commonly employed: **One-vs-One (OvO)** and **One-vs-All (OvA)**. Each approach has its own methodology and application scenarios, making them suitable for different types of classification problems.

1. One-vs-One (OvO) Approach

The One-vs-One (OvO) approach involves creating a binary classifier for every possible pair of classes. This method is particularly advantageous when the number of classes is relatively small, as it allows for a focused comparison between pairs of classes. Each classifier is responsible for distinguishing between two specific classes, effectively ignoring the others.

Implementation:

1. Training Phase:

- For each pair of classes, a binary SVM classifier is trained. For instance, if there are three classes (A, B, and C), the OvO approach will train classifiers for (A vs B), (A vs C), and (B vs C).
- Each classifier is trained using only the data points belonging to the two classes it is meant to distinguish.

2. Prediction Phase:

- During prediction, each binary classifier provides a vote for one of the two classes it was trained on.
- The final class prediction for a given instance is determined by a majority vote among all the classifiers.
- This voting mechanism ensures that the class with the most votes is selected as the final output.

2. One-vs-All (OvA) Approach

The One-vs-All (OvA) approach, also known as One-vs-Rest, involves training a single binary classifier for each class. In this method, each class is treated as the positive class, while all other classes are grouped together as the negative class. This approach is straightforward and scales linearly with the number of classes.

Understanding One-Class Support Vector Machines

Support Vector Machine is a popular supervised machine learning algorithm. It is used for both classifications and regression. In this article, we will discuss One-Class Support Vector Machines model.

One-Class Support Vector Machines

One-Class Support Vector Machine is a special variant of Support Vector Machine that is primarily designed for outlier, anomaly, or novelty detection. The objective behind using one-class SVM is to identify instances that deviate significantly from the norm. Unlike other traditional Machine Learning models, one-class SVM is not used to perform binary or multiclass classification tasks but to detect outliers or novelties within the dataset. Some of the key working principles of one-class SVM is discussed below.

- **Outlier Boundary:** One-Class SVM operates by defining a boundary around the majority class (normal instances) in the feature space. This boundary is constructed to encapsulate the normal data points, effectively creating a region of normalcy.
- **Margin Maximization:** This algorithm strives to maximize the margin around the normal instances, allowing for a more robust separation between normal and anomalous data points. This margin is crucial for accurately identifying outliers during testing.
- **High sensitivity:** One-Class SVM has an in-built hyperparameter called "nu," representing an upper bound on the fraction of margin errors and support vectors. Fine-tuning this parameter influences the model's sensitivity to outliers.